Exact Born sampling of a 70-qubit random-graph-state advantage experiment on one CPU node

Chaitanya Talapalli — independent researcher — ORCID 0009-0000-8811-3349 — chayt128@gmail.com

Code, samples and raw outputs: https://github.com/chayworks/exact-graphstate-sampling (Apache-2.0).

Draft technical note, not submitted. Numbers in braces {...} are pending; every other number comes from a committed artifact listed in the appendix.

Abstract

IBM and the University of Chicago proposed sampling random graph states in a non-stabilizer product basis as a verifiable quantum-advantage task. They ran a 70-qubit, depth-70 instance with an estimated fidelity of 0.37 and extrapolated classical costs of 10^{25} s (matrix product states) and 10^{7} s per sample (extended stabilizer simulation). We show that this instance admits **exact sampling from the ideal Born distribution** on a single 16-core CPU node: **13.0 s per sample**, with 1,012 samples drawn and two complex64 tables of 2^{31} entries.

The method combines two standard ingredients:

- reduction to a graph state with a dynamic program whose state space is set by the cut rank (the rank-width simulability of graph-state measurements);
- a marginal identity that makes the right environment trivial: after the first t qubits are measured, the remaining states Z^v|G⟩ are orthonormal, so Pr(b₁…b_t) = 2^{-t} Σ|P_t|².

Each sample is therefore drawn in two passes, with no rejection and no truncation. This differs from capped rejection sampling (#260 on the tracker), whose output distribution departs from the Born law.

We verify the sampler against dense statevectors on small instances, and on the full instance against an independent amplitude algorithm. An in-place variant draws an exact sample in about 17 s on a laptop with an 8.6 GB table.

We also score IBM's 1,726 published hardware shots exactly: linear XEB **0.466 ± 0.031** (controls at zero), with every per-shot probability agreeing with #260's independent code to 10^{-5} in $\log_2$. Two fidelity estimators built on the exact probabilities give 0.46 ± 0.04 and 0.48 [0.43, 0.54], above the 0.37 that IBM reports from direct fidelity estimation of the post-selected shots (95% lower bound 0.092), and far narrower.

Finally, we map the **hardness frontier** of the construction. The width that governs the attack grows as n/2 − 3.5, so memory doubles every two qubits. At depth n the attack needs one server at n = 70, a large supercomputer at n ≈ 90, and more than any existing machine at n ≳ 100. That locates the size at which this construction escapes rank-width methods.

1. The task

Circuit `nq70_depth70_checks27_basis_fstate`, in the Quantum Advantage Tracker (issue #151):

1. Hadamards on 70 qubits in a line.
2. Brickwork of nearest-neighbour CZ gates to CZ depth 70, with a random √X or S√X on every qubit between layers.
3. On every qubit, $T^\dagger \cdot H \cdot Rz(\theta) \cdot H$ with $\theta = \arccos(1/\sqrt{3})$, a rotation into the "face-state" basis that maximizes stabilizer extent.
4. Measurement in Z.

The hardware run (IBM Boston, Heron R3, with 27 ancillas for spacetime Pauli checks) reports a post-selected fidelity of 0.37 from direct fidelity estimation of the underlying graph state, with a 95% lower bound of 0.092. 1,726 post-selected shots are published with the instance.

2. Method

Graph form. The Clifford part maps to a stabilizer tableau, and then, by Gaussian elimination and local Cliffords, to $|S\rangle = \bigotimes U_j^\dagger |G_A\rangle$. Then

$\langle b|\psi\rangle = 2^{-n/2} \Sigma_x \Pi_j M_j[b_j, x_j] (-1)^{\Sigma_{i<j} A_{ij} x_i x_j}$, where $M_j = W_j U_j^\dagger$.

Cut-rank dynamic program. Fix a vertex order. After the prefix P is processed, the future depends only on $s = A[F,P] \cdot x_P \in GF(2)^{|F|}$. That vector lives in a space of dimension $r = \text{rank } A[F,P]$, so the prefix is a table P[c] of 2^r complex numbers in a chosen basis of that space. Each step is a surjective GF(2)-linear map $(c, x_v) \mapsto c'$. We evaluate it in gather form: a `pdep` scatter plus a kernel of dimension at most 2.

Exact sampling. With prefix P measured with outcomes b_P, the post-measurement state is

$2^{-|P|/2} \Sigma_c P[c] Z^{B c} |G_F\rangle$.

For $v \neq 0$, Z^v has no X part, so it anticommutes with some graph stabilizer and $\langle G|Z^v|G\rangle = 0$. The states $\{Z^v|G_F\rangle\}$ are therefore orthonormal, and $\Pr(b_P) = 2^{-|P|} \Sigma_c |P[c]|^2$. Measuring qubit by qubit in the DP order:

1. compute both candidate extensions' norms (one read-only pass);
2. draw the outcome;
3. store the chosen extension (a second pass).

This is the graph-state analogue of perfect MPS sampling (Ferris & Vidal 2012). The output is the exact Born distribution, up to floating-point error.

Cost for this instance. A greedy-plus-local-search order (`graphsamp/order2.py`) reaches maximum cut rank 30, the value #260 also reports; the natural order gives 32. The production sampling run used an earlier width-31 order and cost $4 \cdot \Sigma_k 2^{r_k} = 6.0 \cdot 10^{10}$ table updates per sample with two complex64 tables of 2^{31} entries; the later runs (laptop, hardware-shot scoring) use the width-30 order.

3. Results

1,012 samples in 13,202 s on an AWS r7a.4xlarge (16 cores, AMD EPYC 9R14), peak RSS 33.6 GB, about $4.50 of on-demand time.

- All samples are distinct.

- Linear XEB against their own exact probabilities is 1.020 ± 0.047, i.e. $2^{70} \Sigma p^2 \approx 2.02$, consistent with Porter–Thomas-like output.

Laptop. With an in-place variant of the program, the arena is a single complex64 table of 2^30 entries (8.6 GB) for a width-30 order. On a laptop (AMD Ryzen 9 9955HX, 16 threads, container memory capped at 10 GB), 3 exact samples took 55 s including allocation, i.e. **about 17 s per exact sample** (`graphsamp/results/laptop_samples_w30.txt`). The in-place sampler was validated on every bitstring of small circuits. On the full instance it reproduces the cloud samples' $\log_2 \Pr$ to $3 \cdot 10^{-7}$. The independent amplitude engine confirms the first laptop sample: −68.5019327 against −68.5019326 reported (`graphsamp/results/xcheck70e_laptop.txt`).

A 76-qubit instance, sampled. To test the frontier with a measurement rather than a projection, we generated a 76-qubit, CZ-depth-76 instance of the same construction (`graphsamp/frontier_instance.py`, seed 1076, 1,465 edges), found an order of width 33 (natural order 38), and ran the in-place sampler on the same 16-core node class (r7a.4xlarge, 128 GiB): **19 exact samples in 3,604 s, 190 s per sample, 67 GB peak** (`graphsamp/results/nq76/`). The node first reproduced the two 70-qubit reference probabilities as a gate. All 19 samples are distinct; their self-consistency XEB is 0.83 ± 0.22. The independent sliced amplitude engine, built with complex64 tables and itself gated on the two known 70-qubit values (−69.5085289 and −68.8761655 against −69.5085302 and −68.8761651), recomputes the first two samples' $\log_2 \Pr$ as −74.1389442 and −75.3402128 against the sampler's −74.1389441 and −75.3402127 (`results/nq76/analysis.txt`). An 80-qubit instance (seed 1080) has an order of width 35, a 275 GB table; its inputs are prepared and the run waits on cloud capacity.

Scoring IBM's hardware shots. The sampler's probability mode evaluates the exact Born probability of a given bitstring through the same code path as sampling. We scored all 1,726 hardware shots published with #151 on an r7a.4xlarge: 6.9 s per shot, 3 h 19 min in all, 16.8 GB peak (`graphsamp/results/ibmxeb/`). The run first reproduced two earlier cloud samples' $\log_2 \Pr$ to $5 \cdot 10^{-7}$ as a gate.

strings	N	linear XEB
IBM hardware shots	1,726	**+0.466 ± 0.031** (bootstrap 95% [+0.41, +0.53])
uniform random strings (control)	64	+0.16 ± 0.12
IBM shots with the bit order reversed (control)	64	−0.15 ± 0.10

Our $\log_2 \Pr$ agrees with #260's independently written FP64 code on every one of the 1,726 strings, to $9.4 \cdot 10^{-6}$ at worst (median $2.8 \cdot 10^{-7}$), and the XEB of +0.4660 is the same to four digits. Two programs written independently assign the same probability to each of IBM's shots.

Normalising by the ideal value $2^{70}\Sigma p^2 - 1 = 1.020 \pm 0.047$, measured on our own exact samples, gives a fidelity estimate F_XEB = 0.457 ± 0.037. A second estimator uses the whole distribution of $z = 2^{70} \cdot p(x)$ over the shots rather than its mean. If each shot is drawn from the ideal distribution with probability F and uniformly otherwise, the density of z over shots is (Fz + 1 − F)e^{−z}, and the maximum-likelihood fit gives F_MLE = 0.48 [0.43, 0.54] (profile likelihood, 95%). The mixture fits the observed shot distribution (Kolmogorov–Smirnov distance 0.018, 5% threshold 0.033; `graphsamp/ibm_fidelity_mle.py`, which recovers F = 0, 0.4 and 1 on synthetic data).

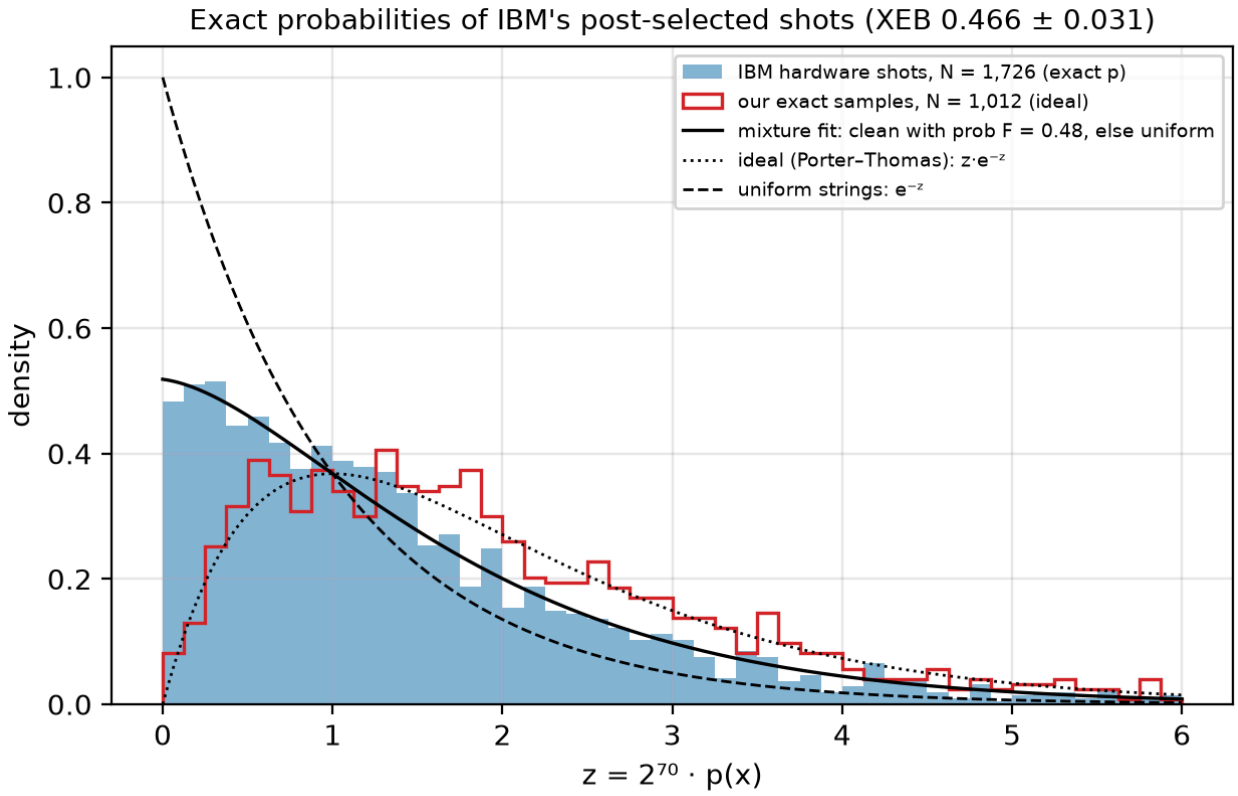


Figure 2. Distribution of $z = 2^{70} \cdot p(x)$ over IBM's 1,726 post-selected shots (exact p), our 1,012 exact samples, and the fitted clean/uniform mixture (`paper/make_figures.py`).

The instance README (tracker repository, `random_graph_sampling/README.md`) states that "XEB requires classical simulations for the outcome probabilities, hence can only be extrapolated for the full-depth circuit". Here it is computed exactly for every published shot. The shots are post-selected by spacetime Pauli checks on 27 ancillas (IBM Boston, Heron R3), and the README reports a post-selected state fidelity of 0.37 by direct fidelity estimation with 80 random stabilizers, "bounded above 0.092 with 95% confidence". Our two estimates, 0.457 ± 0.037 and 0.48 [0.43, 0.54], are consistent with that bound and higher than the point estimate, with a much narrower interval. The quantities differ (the full circuit including the non-Clifford measurement layer, versus the stabilizers of the graph state; the README argues they agree to first order under Pauli twirling), and linear XEB is known to over-estimate fidelity under non-depolarising noise, so we report the gap without interpreting it.

4. Verification

1. **Small instances.** On circuits of the same form with up to 9 qubits, the distribution implied by the sampler matches the statevector on every bitstring to $2 \cdot 10^{-15}$. The C++ sampler's 20,000-shot histogram has the same total-variation distance from the exact distribution as an ideal sampler's. Every test has a negative control that fails as intended.
2. **Full instance.** A sliced amplitude engine recomputes $\log_2 \Pr$ of sampled bitstrings; it is a different algorithm from the sampler.

sample	sampler $\log_2 \Pr$	independent engine $\log_2 \Pr$
1	−69.5085302	−69.5085294
2	−68.8761651	−68.8761651
3	−68.3318209	−68.3318207
4	−69.0978566	−69.0978571
5	−70.2946875	−70.2946876

Control: sample 1 with 10 bits flipped gives −72.8808316. That is 3.37 lower in $\log_2$ (10× less likely), against agreement within 1e-6 on the real samples.

1. **Reproducibility.** The production run first reproduced, bit for bit, three samples generated on a different machine, compiler and thread count.
2. **Engine-independent check.** While the cut rank of the measured prefix equals its length, the reduced state is maximally mixed, so the prefix marginal is exactly uniform; for the production order that holds for the first 30 steps. Over the 1,012 samples the first 30 step-bits show pair parities (435 tests) and triple parities (4,060 tests) at chance (4.8% and 4.4% beyond 2σ, 4.6% expected; largest |z| 3.2 and 3.8). Of the 30 single-bit means, 4 fall beyond 2σ ($p \approx 0.05$), the largest at 2.4σ. A control in which one bit is replaced by a copy of another fails the test (`graphsamp/check_uniform_prefix.py`, `graphsamp/results/uniform_prefix_check.txt`). This uses only the rank profile and the sample file.
3. **Stabilizer-only conditionals.** Once the cut rank falls below the prefix length, the prefix marginal has a closed form from the stabilizer group alone: Pr(b_F) = 2^{-t} Σ_S sign(S) Π_{j∈supp S} (−1)^{b_j} n_j[S_j], summed over the 2^{t−rank} stabilizers supported on F, with n_j the Bloch vector of the measured operator. We enumerate that sum exactly while the deficit is at most 16 (steps 1–42 of the width-30 order) and compare the resulting conditionals Pr(b_k | b_1…b_{k−1}) with the sampler's chain output for all 1,726 hardware shots: the two agree to $3.3 \cdot 10^{-8}$ (the print resolution), while the conditionals themselves depart from ½ by up to $8 \cdot 10^{-5}$ at those steps (`graphsamp/stabprefix.py`, `graphsamp/results/chain/stabprefix_ibm_fwd.txt`). On small circuits the same formula reproduces the statevector marginals to $6 \cdot 10^{-15}$ on 162 prefixes, and a flipped generator sign fails it. This is a check of the dynamic program's fine structure by a method that never builds the DP table.

The same formula explains where the information in a string lives: a stabilizer of weight w contributes at most max|n|^w, so the conditionals stay within 10^{-4} of ½ until the prefix supports many low-weight stabilizers, which in this order happens only in the last ~10 steps; the per-step spread of the chain conditionals grows by about 1.4× per step from 10^{-4} at step 50 to 0.56 at step 70, and the last 8 steps carry 2.91 of the 2.93 variance of $\log_2$ Pr across the hardware shots (`graphsamp/results/chain/ibm_fwd_chain.txt`). For our 1,012 exact samples the picture is the same: the spread of the conditionals first exceeds 10^{-3} at step 51 (52 in the reversed order), and the last 4 and 8 steps account for 90% and 100% of the variance of $\log_2$ Pr (`graphsamp/results/chain/analysis_profile.txt`).

1. **Three dynamic-program paths agree.** Scoring every string along the width-30 order, along its reverse (the same width, a different table layout and step sequence), and, for our samples, along the width-31 production order gives the same $\log_2$ Pr: the forward and reversed orders agree to $1.4 \cdot 10^{-5}$ at worst (median $4 \cdot 10^{-7}$) on the 1,726 hardware shots and to $3.5 \cdot 10^{-6}$ on the 1,012 samples, and the width-31 run agrees with the width-30 order to $4.3 \cdot 10^{-6}$ (`graphsamp/results/chain/analysis_crossorder.txt`). The stabilizer-only check of item 5 also holds along the reversed order and on the ideal samples ($3.3 \cdot 10^{-8}$, `graphsamp/results/chain/stabprefix_rev_ideal.txt`).

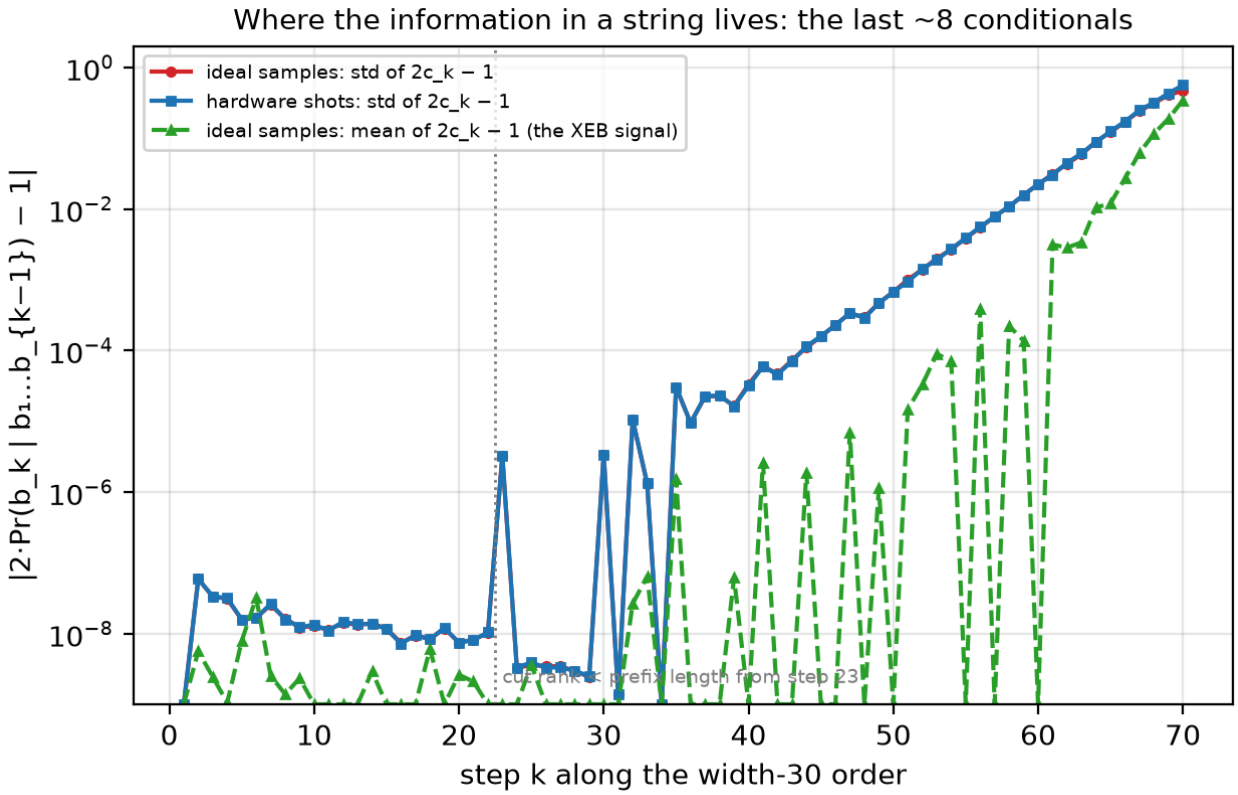


Figure 3. Spread of the exact per-step conditionals along the width-30 order for the hardware shots and the ideal samples, and the mean of 2c_k − 1 over ideal samples (the per-step XEB signal). The first 22 conditionals are exactly ½; the information sits in the last ~8 steps.

Per-step fidelity of the hardware shots. Dividing the hardware mean of 2c_k − 1 by the ideal mean gives a fidelity-like ratio for each informative step. Because the information sits in the last steps, only four steps per order are informative (ideal mean above 0.05): in the forward order, qubits 9, 8, 13 and 50 give 0.45 ± 0.12, 0.54 ± 0.08, 0.52 ± 0.07 and 0.48 ± 0.04; in the reversed order, qubits 45, 68, 53 and 21 give 0.47 ± 0.10, 0.43 ± 0.07, 0.43 ± 0.06 and 0.53 ± 0.05. All eight are consistent with the global fidelity estimates of 0.46–0.48 and none stands out, so the shots show no sign of a qubit that is much worse than the rest among those eight. A per-qubit map of all 70 qubits would need an order ending in each group of about four qubits, roughly 18 further width-30 passes; we did not run them.

5. Hardness frontier

We generated circuits with the #151 construction (linear chain, alternating CZ brickwork with random √X or S√X layers, CZ depth n) for n = 30–120, with one instance per n, and ran the same order search (`graphsamp/frontier.py`, output `graphsamp/results/frontier.jsonl`). At n = 70 the generated instance matches the real one: 1,192 vs 1,150 edges, natural-order width 33 vs 32, best width 31 vs 31 at the same search budget (a longer search on the real instance reaches 30).

n	best width found	memory (2 complex64 tables)	table updates per exact sample
30	12	< 1 MB	2^16.7
50	22	67 MB	2^26.4
60	26	1.1 GB	2^30.8
70	**31**	**34 GB**	**2^36.0**

76 (sampled)	**33**	**69 GB in-place**	**2^38.2**
80	36 (35 found later)	1.1 TB (275 GB in-place)	2^41.2
90	42	70 TB	2^46.3
100	47	2.3 PB	2^51.4
110	51	36 PB	2^55.9
120	56	1.2 EB	2^61.2

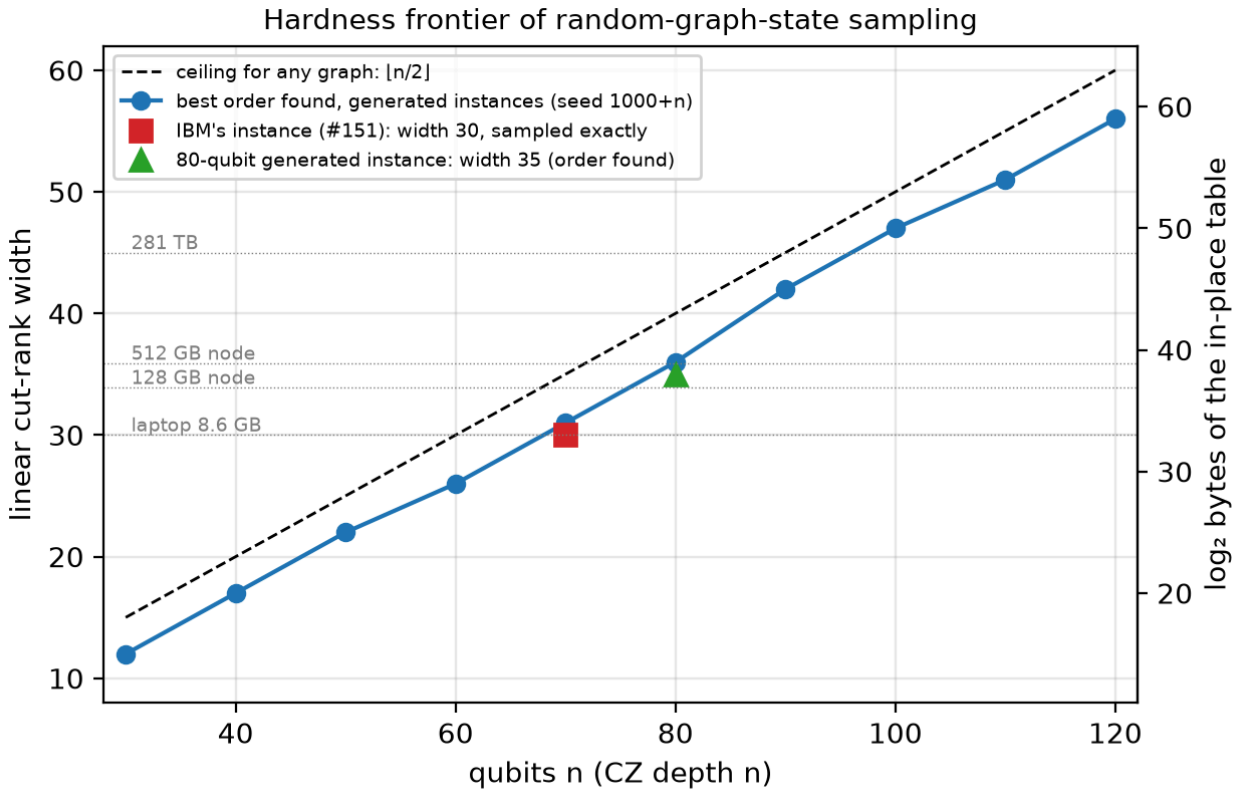


Figure 1. Best width found versus n for the construction, the ⌊n/2⌋ ceiling that holds for every graph, and the memory of the in-place table.

The width fits 0.492·n − 2.9; at every n it is between n/2 − 4 and n/2 − 3. So memory doubles with every two qubits. At depth n, this method moves from one server (n = 70) to one large-memory node (n ≈ 80), to a leadership-class supercomputer (n ≈ 90), and past any existing machine (n ≳ 100).

A bound that holds for every graph. The cut rank of a prefix F is at most min(|F|, n − |F|), so for any graph on n vertices every linear order has width at most ⌊n/2⌋, and the in-place sampler needs at most 2^{⌊n/2⌋} complex64 entries and at most 4·n·2^{⌊n/2⌋} table updates per exact sample, whatever the graph and whatever the product measurement basis. At n = 70 that is 275 GB and about $9.6 \cdot 10^{12}$ updates, i.e. roughly 4 hours per sample at our measured $6.4 \cdot 10^{8}$ updates per second on 16 threads; at n = 76 the table is 2.2 TB (the largest single cloud nodes); at n = 90 it is 281 TB (aggregate memory of a leadership machine); at n = 100 it is 9 PB. So no graph-state sampling experiment with product measurements can escape a single large-memory node below about 76 qubits, or a supercomputer below about 90, no matter how the graph is chosen; the random brickwork construction already sits within 3–4 of this ceiling, so the only lever left to the experimenters is the qubit count.

Caveats. The widths are upper bounds from heuristic orders; better orders or tree decompositions would lower them. Slicing trades memory for time at a measured cost of about 1.5× per sliced vertex on

this instance, not 2×. The table has one instance per n. For a fixed classical budget, then, the safe direction for the experiment is more qubits at full depth. A width of ~n/2 is what random graphs give, and the 70-qubit instance sits about 30 qubits below the point where this attack stops being practical.

6. Related work

- Rank-width simulability of graph-state measurements: Van den Nest, Dür, Vidal, Briegel, PRA 75, 012337 (2007).
- ZX-calculus rank-width contraction: Kuyanov & Kissinger, arXiv:2603.06764.
- Exact amplitudes of IBM's doped circuits by T-injection and rank-width: Renaud, `dcs-rankwidth`; tracker #249.
- Capped rejection sampling for this instance on GPUs: tracker #260.
- Exact tensor-network amplitudes for the doped 70-qubit instance: Manabe, Gu, Pan, arXiv:2608.13110.

Appendix: artifacts

- `graphsamp/results/samples_nq70.txt`, `run.log`, `time_nq70.txt`, `inputs.sha256`, `check55_result.txt`: the production sampling run and its reproduction check.
- `graphsamp/results/xcheck70_independent.txt`, `xcheck70d.txt`, `xcheck70e_laptop.txt`: independent recomputation of five cloud samples, the 10-bit control, and one laptop sample.
- `graphsamp/results/laptop_samples_w30.txt`: the laptop run (3 samples, timing).
- `graphsamp/results/ibmxeb/`: exact $\log_2$ Pr of the 1,726 hardware shots and the controls, timing, gate, input hashes, and `analysis.txt` (XEB, comparison with #260, fidelity estimators).
- `graphsamp/results/uniform_prefix_check.txt`: the engine-independent uniformity check.
- `graphsamp/results/frontier.jsonl`: the frontier search output.
- `graphsamp/README.md`: test table and commands.